

SkiLT: What Agent Reinforcement Learning Actually Learns from a Latent Skill Library

Anonymous submission

Abstract

Retrieved skill libraries improve large language model agents, but rendering skills as text adds hundreds of prompt tokens at every decision step, and reinforcement learning pays that cost on every rollout. SkiLT asks whether the same conditioning survives full-actor, episode-reward RL in a compact latent form, and what such a library then contributes. A Composer initialized from the actor’s own base model compresses each of 44 SkillRL library entries into eight soft vectors, which are installed as retrievable vocabulary rows before the full actor is trained with GRPO. On the complete ALFWorld evaluation sets SkiLT reaches 90.0 *valid-seen* and 86.6 *valid-unseen* success against 87.9/79.9 for a released text-skill RL checkpoint under the same harness, with the skill section reduced from ~ 970 to ~ 140 tokens per step. That checkpoint was trained under a different recipe, so it anchors rather than controls the comparison, and a second SkiLT run reads 87.9/79.9, leaving the out-of-distribution margin unestablished. The mechanism study then revises the reading of this result. The installed rows were effectively frozen during the flagship run (relative drift $\leq 2.4 \times 10^{-6}$): their activation-scale norms starved them under Adam and pushed nearly every update below the bfloat16 storage boundary. A 2×2 factorial isolates both failure modes, and a function-preserving repair restores row learning at RL scale. Yet substitutions drawn from the deployed bank or its empirical statistics, carrying no correct per-entry identity, are indistinguishable from it at the tests’ $\sim \pm 7$ -point resolution, while removing the conditioning costs 21 points and retraining from an isotropic norm-matched bank does not recover. Compact latent conditioning therefore reaches text-skill RL performance under this harness, but we detect no contribution from code identity or per-entry distinctness, and the load-bearing property of the conditioning remains unidentified.

1 Introduction

Skill libraries, curated collections of reusable procedural advice, are a leading recipe for LLM-agent reinforcement learning: SkillRL (Xia et al. 2026) retrieves text skills into every prompt and reports strong ALFWorld results (89.9). Text skills, however, are expensive: the skill section alone adds roughly a thousand tokens to every step’s prompt (Table 3),

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

and RL training pays that tax in every rollout. At this recipe’s scale that is $\sim 4\text{--}7 \times 10^8$ extra prefill tokens per training run (150 updates $\times$ 128 trajectories $\times$ 21–36 steps $\times$ ~ 970 tokens; Table 3). Prefix caching mitigates recomputation at best: it does not shrink the KV footprint, the context-budget share, or the per-episode latency of Table 3, and the cost grows with the library, overflowing the prompt budget when the bank expands mid-training (§5).

A natural fix is to compress skills into continuous representations. Concurrent work confirms the appeal and the current limits: under their own protocols, supervised soft-skill prefixes trail text skills on ALFWorld (71.6 vs. 79.1) (Tao et al. 2026), and hypernetwork-generated weight-space skills on a frozen backbone stay well below RL-level performance (74.3/69.4 *valid-seen/valid-unseen*) (Yu et al. 2026); our own supervised stage is no exception (52.9 for the flagship Stage-1 objective, §4). Latent-memory systems that do use reward differ along axes worth separating: which component is updated, what supplies the reward, and whether the consuming policy trains at all (§2). We did not find a prior evaluation that carries a persistent retrieved library through full-actor, episode-reward agent RL and measures it against an RL-trained text-skill agent on the same benchmark and harness.

SkiLT supplies that evaluation. A Composer (the actor’s own base model plus k learned query vectors) encodes each library entry into $k=8$ vectors, which are installed in the actor’s vocabulary as new SKILL token rows (LM head untied and zeroed); retrieval is unchanged from SkillRL, but each retrieved skill enters the prompt as k tokens instead of hundreds of words. Building the Composer from the actor’s own base model keeps the codes near its representation space; entries encoded by a mismatched body prove destructive (§4.4). The full actor is then trained with GRPO for 150 epochs under SkillRL’s hyperparameters. On the complete ALFWorld sets SkiLT reaches 90.0/86.6 *valid-seen/valid-unseen* against 87.9/79.9 for the released text-skill checkpoint, at 45% fewer prompt tokens per episode. Because that checkpoint was trained under a different recipe, we treat it as an external anchor: informative about where compressed conditioning lands, not a controlled test of representation. A second SkiLT training run reads 87.9/79.9 on the paired protocol, so we report both runs and rest no claim on the out-of-distribution margin of either alone (§4).

The paper’s second half asks what the reward actually

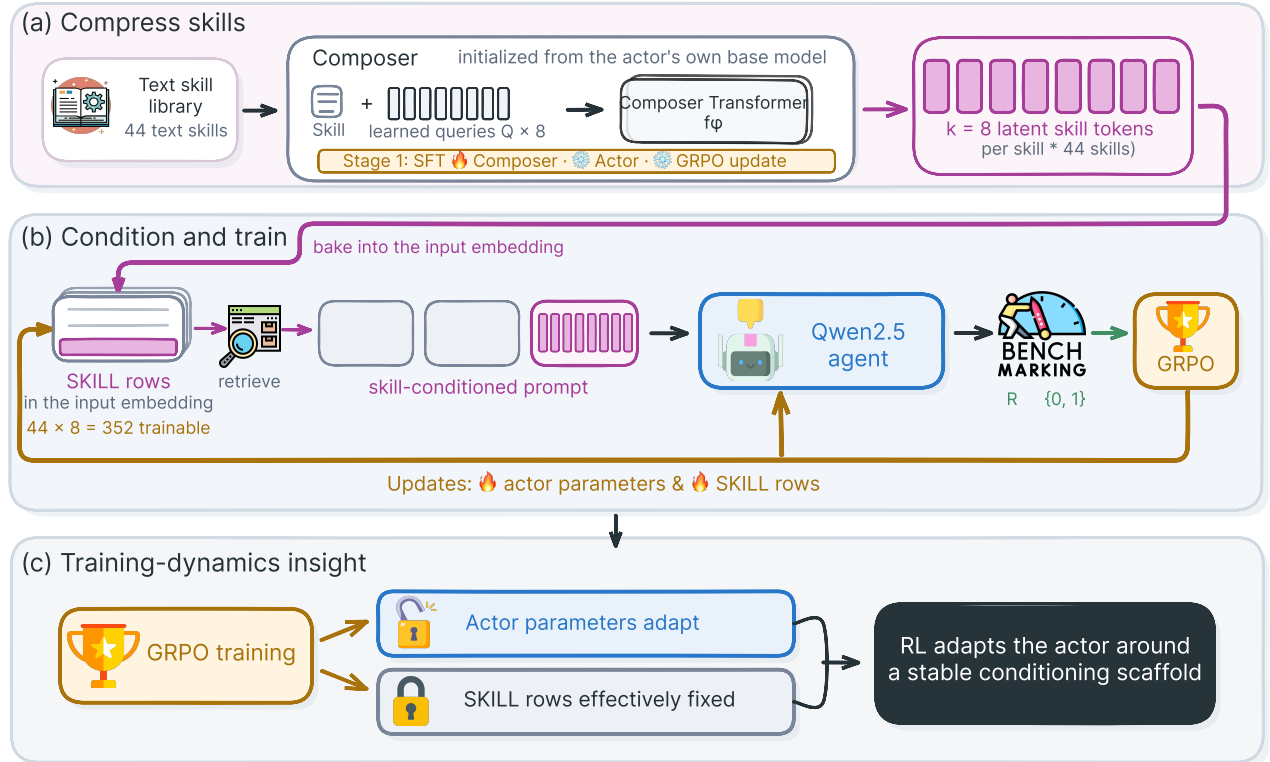


Figure 1: SkillT in three stages. (a) A Composer initialized from the actor’s own base model encodes each library entry into $k=8$ latent vectors, supervised with the actor frozen. (b) The vectors become 352 SKILL rows in the actor’s input embedding, so a retrieved skill costs 8 tokens instead of hundreds of words, and GRPO trains the full actor on episode reward. (c) The audit of §4.3: the actor adapts around rows that stay effectively fixed.

trained, and the answer revises the natural reading of such a system. The installed rows were effectively frozen: after 150 epochs their relative drift is at most 2.4×10^{-6} , 620 times less than ordinary vocabulary rows, because activation-scale row norms both starve the rows under Adam and push their updates to the bfloat16 rounding boundary (Mickevičius et al. 2018; Miah and Belilovsky 2026). A 2×2 factorial isolates the two mechanisms, a function-preserving repair restores learning, and a full repaired-bake RL run verifies it at scale. Interventions then find no measurable dependence on skill identity, or even on the codes being distinct: one code repeated 44 times matches the deployed bank within test resolution, as does a surrogate sampled from that bank’s empirical moments, while an isotropic bank at matched norm lands 28.6 points lower. Re-encoding entries with a mismatched encoder body is actively destructive, a failure mode text libraries cannot have.

Contributions.

- **Compressed conditioning carried through agent RL, measured on complete evaluation sets.** We report exact full-set outcomes and token accounting for an actor whose retrieved library enters as 8 soft vocabulary rows per entry, against a released text-skill RL checkpoint and a no-library control under one harness (two SkillT training runs; three decoding evaluations per checkpoint). The skill section falls from ~ 970 to ~ 140 tokens per step (§4).

- **An anatomy of what agent RL trains in such a library.** We document a silent optimization failure in which activation-scale vocabulary rows receive no effective update, isolate its two mechanisms with a controlled 2×2 factorial, and verify a function-preserving repair in a full 150-epoch RL run whose rows demonstrably move. Interventions then map what the frozen rows contribute. Permutation, independent-draw substitution, collapsing the library to one repeated code, and a random draw matched to its first two moments are all indistinguishable from the deployed library at $\sim \pm 7$ -point resolution, while removal, corruption, and retraining from isotropic noise all fail. Every working substitution was built from the deployed bank or its statistics, so mean direction, covariance, anisotropy and Composer provenance stay confounded, and the minimal load-bearing geometry is unresolved (§4.3–§4.4).

A leakage-free replication of self-evolving skill libraries, which reproduces neither arm’s reported gains, appears as a case study in §5.

2 Related Work

Skill and memory libraries for agent RL. One line of work augments agent RL with retrieved *text* knowledge: SkillRL’s recursive skill evolution (Xia et al. 2026), graph-structured libraries (Li et al. 2026), unified skill-lifecycle training (Shi et al. 2026; He et al. 2026b), and episodic-

memory RL (Zhang et al. 2026a); a second line *internalizes* skills into weights and discards them at inference (He et al. 2026a; Lin et al. 2026). Appendix E surveys the wider 2026 literature. These systems keep skills as text or erase them; we keep the library explicit but continuous and carry it through the same RL recipe. The text-side evidence also sharpens our dynamics case study: Zhang et al. (2026b) report near-zero mean marginal utility for ungated LLM-generated skills and He et al. (2026b) show that unvalidated adoption collapses training; the released mechanism we port has no admission gate, and §5 measures the consequence.

Compressing prompts and skills into soft tokens. Gist tokens (Mu, Li, and Goodman 2024), AutoCompressor (Chevalier et al. 2023), ICAE (Ge et al. 2024), and 500xCompressor (Li, Su, and Collier 2024) compress context with supervised or self-supervised objectives. TokMem stores procedures as single trainable memory tokens under a frozen backbone (Wu, Hao, and Mou 2025), and Concept Tokens learn one vocabulary row per concept (Sastre and Rosá 2026); these are the single-token ancestors of our mechanism, without agents or reinforcement learning. Closest in mechanism, Skill Neologisms optimizes new vocabulary soft tokens against a frozen model for continual learning and shows that independently learned skills compose zero-shot in context (Berthon, Astorga, and van der Schaar 2026); its tokens are learned from skill-centered data rather than compressed from a curated library, on a frozen consumer and single-turn tasks, with no retrieval and no reinforcement learning. Classical soft-prefix methods (Lester, Al-Rfou, and Constant 2021; Li and Liang 2021) are the correct ancestors of the mechanism our controls actually support: because permuting skill identities and substituting an independently trained library produce no detectable change (§4.3), a global learned prefix is a live alternative explanation for part of the effect. Concurrently, SKIM (Wang et al. 2026) compresses procedural skills at multiple resolutions and SoftSkill (Tao et al. 2026) tunes soft skill prefixes; both are supervised, and SoftSkill’s latent skills underperform text on ALFWorld. SkillT carries this representation through environment-reward RL and evaluates it against an RL-trained text-skill agent under a shared harness.

Latent skills and memories with reward signals. Several concurrent systems connect latent memory to reward, differing from ours along identifiable axes rather than in kind. ELM trains per-instance soft-prompt memories with GRPO under self-consensus pseudo-rewards, its consumer frozen (Dorovatas, Kalaj, and Aljundi 2026); LatentMem’s LMPO propagates task-level signals through a memory composer with the executor frozen (Fu et al. 2026); MemGen weaves fresh latent memory per state with no persistent bank (Zhang, Fu, and Yan 2025); ElasticMem keeps its bank fixed and GRPO-trains retrieval, budgeting and a consumer-side LoRA, so its stored latents receive no reward gradients (Feng et al. 2026); Neural Procedural Memory steers with persistent implicit activations, training-free (Zhao et al. 2026); Soft Tokens, Hard Truths applies RL to ephemeral continuous chain-of-thought (Butt et al. 2025); and LatentSkill compiles skills to LoRA adapters supervisedly (Yu

et al. 2026). We did not find a prior evaluation combining a persistent retrieved library, a fully reward-trained actor, and a same-harness comparison against text-skill RL; we state that as a search result, not as priority.

Latent reasoning and training numerics. Continuous-representation reasoning (Hao et al. 2025; Zhu et al. 2025) and RL over latent thoughts (Deng et al. 2026) argue that continuous tokens carry denser information than text; our controls temper that premise for libraries, since what survives RL here is task-general conditioning rather than per-skill addressable content (§4.3). On the numerics side, mixed-precision training classically requires full-precision master weights because updates below the storage format’s rounding boundary vanish (Micikevicius et al. 2018), and in RL fine-tuning most per-step updates are already invisible after the bfloat16 cast (Miahi and Belilovsky 2026); our frozen rows are an extreme in-the-wild instance, with an added scale-mismatch mechanism specific to installed activations.

3 Method

3.1 Setting

Following SkillRL (Xia et al. 2026), an actor π_θ (Qwen2.5-7B-Instruct (Qwen Team et al. 2025)) solves ALFWorld (Shridhar et al. 2021) household tasks; a library $\mathcal{L} = \{s_1, \dots, s_{55}\}$ is retrieved per task and injected into each step’s prompt. $\mathcal{L}$ is SkillRL’s released initial skill bank, used verbatim by both arms: 44 skills (12 general, 32 task-specific) plus 11 common-mistake advisories that carry a different schema in the released file, matching the 55-entry initial library reported in their paper. At step t the prompt x_t assembles the task goal, the last 10 interaction steps, and the retrieved skill set $\mathcal{S} \subset \mathcal{L}$; the actor emits a thought and an action $a_t \sim \pi_\theta(\cdot \mid x_t)$, and an episode terminates with reward $R \in \{0, 1\}$ (task success) within 50 steps.

3.2 Composer: skills to latent tokens

For each library entry s_i (text), the Composer computes $z_i = f_\phi([E(s_i); Q])[-k:] \in \mathbb{R}^{k \times d}$, where f_ϕ is a transformer initialized from the actor’s own base model, E its input embedding, and $Q \in \mathbb{R}^{k \times d}$ are learned query vectors appended to the skill text; the last k hidden states form the skill’s latent code. The appended-query, last- k readout follows the gist-token lineage (Mu, Li, and Goodman 2024); we did not explore pooling alternatives, and the Composer is deliberately not the component RL optimizes. Building it from the actor’s own base model also avoids a cross-model projection, which underperformed in an early pilot that pre-dates a parser fix (Appendix A reports it as historical rather than controlled). Encoding is one-shot per skill, so the larger Composer adds no per-step cost. Stage 1 trains the Composer body and Q jointly with a supervised objective on SkillRL’s SFT corpus: teacher-forced next-action prediction with latent codes substituted for the skill text, while the consuming actor is frozen (no actor parameter enters the optimizer; Appendix A resolves an integrity alert in these logs as a floating-point reduction artifact). We fixed $k=8$ before RL from an exploratory pre-RL sweep over $k \in \{1, 2, 4, 6, 8, 16, 32\}$ in which training-draw variance dominates the k effect; $k=8$ is

an accuracy-cost operating point, not a tuned optimum, and we make no claim about how the RL endpoint responds to k (one full run per value). LatentMem’s sensitivity study settles on the same length for the same reason (Fu et al. 2026). Appendix A gives the sweep, the hyperparameters, and a later Stage-1 objective revision that the flagship predates.

3.3 Baking and RL

Each skill’s k vectors are added to the actor’s vocabulary as new rows $\{\text{SKILL}_{i,1..k}\}$ (input embedding $\leftarrow z_i$; untied LM head rows zeroed, which discourages emitting the new tokens without guaranteeing it; evaluation additionally masks these 352 IDs explicitly, and training does not). Retrieval (unchanged from SkillRL: top-6 general + task-matched + 5 mistakes) injects each selected skill as k tokens. The flagship bakes the 44 skill entries as latent rows; the 11 mistake entries carry a different schema and were silently skipped by the bake extractor, a defect found post hoc, so they enter the latent arm’s prompt as inert placeholder strings while the text arm renders them as readable advice, an asymmetry conservative against our method (§4.4 gives the corrected encoding). The full actor is then trained with GRPO (Shao et al. 2024) on episode reward for 150 epochs in the verl framework (Sheng et al. 2024), matching SkillRL’s optimization scalars (lr 10^{-6} , batch 16, group 8, KL 0.01; Table 1 lists what is *not* matched). Schematically,

$$J(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{g=1}^G \frac{1}{|\tau_g|} \sum_t \min(\rho_{g,t} \hat{A}_g, \text{clip}(\rho_{g,t} \hat{A}_g)) - \beta \mathbb{D}_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}}) \right], \quad (1)$$

where $\rho_{g,t}$ is the token-level importance ratio and $\hat{A}_g$ the group-normalized episode advantage over $G=8$ rollouts of the same task. The implementation adds an invalid-action penalty before group normalization, dual clipping, an entropy term and a low-variance KL estimator, all token-mean aggregated; Appendix H gives the exact objective. Architecturally, the skill rows are ordinary embedding parameters and $\nabla_{z_i} J$ is defined; whether they *effectively* train is a numerics question we return to in §4.3.

3.4 The numerics of training installed rows

Installed rows are Composer hidden states, with norms of ~ 235 against ~ 0.9 for ordinary vocabulary rows. Activation scale is required where the rows are *consumed*, since they substitute for hidden states; the defect is storing them at that scale, which has two consequences under Adam at lr 10^{-6} . Adam steps are approximately lr-sized per element whatever the parameter scale, since the gradient is divided by its own running RMS, so a row at $\sim 260\times$ the norm makes $\sim 260\times$ less relative progress (*starvation*); and bfloat16’s 7-bit mantissa puts the unit in the last place at 0.03 for elements of magnitude ~ 4 , so those updates round to zero when stored (*erasure*) (Mickevicius et al. 2018; Miah and Belilovsky 2026). The repair is a per-row power-of-two rescaling at bake time, storing rows at ordinary scale and multiplying back at consumption so the round trip is bitwise exact and the function unchanged at initialization, together with full-precision storage. The flagship predates it; §4.3 isolates both halves factorially and measures the consequences.

	SkiLT	released text
base model, environment, reward		identical
GRPO scalars (lr, batch, group, KL)		identical
harness, games, T , parser		identical
library during <i>training</i>	static 44	evolving
prompt budget during training	4,096	6,000
entries rendered readably	44 of 55	55 of 55
training runs available	2	1 (external)

Table 1: What the headline comparison does and does not hold fixed. The released checkpoint was trained with validation-driven library evolution and a larger prompt budget, and its evaluation renders 11 mistake advisories our bake could not encode (§3), an asymmetry favouring the text arm.

4 Experiments

Our experiments ask where a compressed library lands on complete evaluation sets against a no-library control and an RL-trained text-skill agent, what reward actually trained inside it, and what the representation costs and requires; §5 adds a self-evolution case study.

4.1 Setup

Following SkillRL (Xia et al. 2026): ALFWorld, its released skill bank, retrieval policy and GRPO recipe (Appendix H).

Exact full-set evaluation. The standard verl harness samples evaluation games with replacement (97 distinct of 140 slots); we shard one game per worker so each is evaluated exactly once (140 *valid-seen*, 134 *valid-unseen*; $T=0.4$, terminal checkpoints, no validation selection). Cells noted[§] use a sequential single-GPU harness with the same environment, parser and sampling, which additionally records per-episode outcomes. Its game order is fixed by a seeded draw, so cells compare position by position; because the records store no game identifiers, pairing is verified by task-type sequence and we treat these McNemar tests as exploratory sensitivity analyses rather than confirmatory matched-pairs inference (Appendix B). Cross-harness deviations reach ~ 5 points with no consistent direction, so each estimate carries a test computed *in its own protocol*. Endpoint cells decode from their checkpoint’s stored rows; intervention cells (§4.3–§4.4) overwrite or extend the 352 rows on the fixed flagship checkpoint.

What is and is not matched. The text arm is SkillRL’s released checkpoint, not one we trained; Table 1 states the differences; the comparison is an external anchor, and representation is not the only variable separating the two. A matched static-text retrain is the experiment this paper most needs; we spent the available runs on the mechanism study and the dynamics replication instead.

Selection disclosure. The Stage-1 library carried into RL was chosen on its pre-RL *valid-seen* evaluation, so *valid-seen* is not an untouched test split for SkiLT; *valid-unseen* is.

Appendix B details both harnesses and the McNemar replicates; Appendix H is the run ledger.

4.2 Where Compressed Conditioning Lands

Table 2 reads as the argument chain. The library matters under this recipe: removing it and rerunning the same GRPO

training costs 15.7 points *valid-seen* and 25.4 *valid-unseen*, mostly out of distribution. That value is not free from text at this scale: as text the library adds ~ 970 prompt tokens per step while moving an untrained actor only from 42.1 to 44.3. Supervised compression alone does not reach RL-level performance either, ours included (52.9 frozen-actor, mean 45.1 ± 6.9 over six draws). Carrying the compressed library through RL closes most of that gap: on the exact full-set protocol SkiLT reaches 90.0/86.6 against 87.9/79.9 for the released text-skill checkpoint (+2.1 *valid-seen*, +6.7 *valid-unseen*, 116/134 vs. 107/134) at 45% fewer prompt tokens per episode.

Within the sequential protocol, which records per-episode outcomes and supports exact paired tests, the same checkpoints give 126/140 vs. 120/140 *valid-seen* (discordant 14:8, exact McNemar $p=0.29$) and 123/134 vs. 101/134 *valid-unseen* (discordant 24:2, $p=1.05 \times 10^{-5}$). Two further decoding evaluations of the same two checkpoints reproduce both outcomes ($p \leq 7.3 \times 10^{-4}$ *valid-unseen*, never below 0.11 *valid-seen*). These are repeated evaluations of one checkpoint pair, not independent training runs.

Training-run variation is the larger source of uncertainty, and we have two runs rather than the three or more a method-level claim needs. A second SkiLT run (same nominal recipe, environment seed 1) reads 87.9/79.9[§] against the first run’s 90.0/91.8[§]: no detectable *valid-seen* difference (paired $p=0.63$) but 11.9 points lower *valid-unseen* (discordant 19:3, $p=8.6 \times 10^{-4}$). Against the text checkpoint the second run keeps the *valid-unseen* direction without significance (107/134 vs. 101/134, 18:12, $p=0.36$). No *valid-seen* difference was detected in any comparison; the *valid-unseen* difference is large and highly significant for one run and small and non-significant for the other, so the *magnitude* of any out-of-distribution advantage is not established by two runs. One further caveat cuts against us: the only text-skill agent we trained *ourselves* (the leakage-free dynamic arm of §5) reaches 77.1/78.4 with its base library, some 11 points below the released checkpoint in distribution. We attribute that to the evolution defect and to its truncation at step 130, but cannot rule out that our pipeline is simply less favourable to the text representation. The seen-to-unseen degradation differs sharply by condition: SkiLT loses 3.4 points, the text checkpoint 8.0, and the no-library run 13.1.

4.3 What Did RL Actually Train?

The rows were effectively frozen. Comparing the trained checkpoint’s 352 skill rows against the baked values, mean relative drift is 7.2×10^{-7} (maximum 2.3×10^{-6}), 620 times below the 4.5×10^{-4} of ordinary vocabulary rows, and the 0.2% of changed elements each moved exactly one bfloat16 rounding step. Attention weights trained normally, and the zero-initialized LM-head skill rows moved off zero. The gains are therefore dominated by actor adaptation *around* an effectively fixed conditioning bank. This does not weaken the headline comparison, which stays symmetric (text skills are equally fixed during text-skill RL), but it converts the intended claim of reward-trained skill contents into a numerics finding.

Condition	runs	<i>valid-seen</i> $\uparrow$	<i>valid-unseen</i> $\uparrow$
<i>After RL (same recipe, same harness, same base model)</i>			
no library	1	74.3	61.2
SkiLT (soft rows)	2	90.0	86.6
2nd training run [§]		87.9	79.9
repaired bake [§]	1	87.1	85.8
random bank [§]	1	61.4	52.2
<i>External anchor (different recipe)</i>			
SkillRL released text ckpt	n/a	87.9	79.9

Table 2: ALFWorld success (%) on the complete evaluation sets (140 *valid-seen*, 134 *valid-unseen*), all cells measured by us. Pre-RL conditions appear in Table 3. The random-bank row replaces the library with norm-matched Gaussian codes and retrains under the same nominal recipe (§4.5; exploratory, Appendix G). “runs” counts independent RL training runs; the *valid-unseen* split is untouched by our selection decisions, *valid-seen* is not (§4.1). [§]sequential harness. The released text checkpoint differs in training history (Table 1), so it anchors rather than controls the comparison. Appendix C lists externally reported numbers under their own protocols.

	succ.% [§]	skill/step	steps	prompt/ep	gen/ep	gen s
no-skill	42.1	0	36.4	30.2K	4,170	120.6
text skills	44.3	~ 970	36.1	65.0K	4,385	118.6
soft rows	52.9	~ 140	32.9	31.6K	1,328	35.7
text-RL ckpt	85.7	~ 970	20.9	32.7K	1,094	28.4
SkiLT RL	90.0	~ 140	18.5	17.9K	915	25.0

Table 3: Sequential-harness accounting (*valid-seen*); all columns, success included, are sequential[§] measurements (text checkpoint 85.7 here vs. 87.9 sharded in Table 2). The representation effect is the *skill/step* column. The per-episode columns are jointly behavioural: the 45% prompt reduction after RL also reflects shorter episodes (18.5 vs 20.9 steps). “gen s” times generation only, not end-to-end latency.

Factorial cause and fix. Table 4 isolates the two mechanisms of §3.4 with a controlled study on the baked actor (only the 352 rows trainable, 300 steps, $\text{lr } 10^{-6}$; NaN-poisoned base rows guarantee the wiring). Neither full-precision storage alone (A2) nor rescaling alone (A3) restores learning; both together (A4) bring the applied-to-intended update ratio to 1.0 and move held-out CE. A full 150-epoch RL run with the A4 bake verifies this at scale: relative drift 2.7×10^{-3} at the endpoint with 45.8% of row elements crossing the bfloat16 boundary, against 7.2×10^{-7} and 0.2% in the flagship. The task endpoint reads 87.1/85.8[§] against the frozen-rows flagship’s 90.0/91.8[§]; no difference is detected in distribution (paired $p=0.50$) and the 6.0-point *valid-unseen* gap is not significant either (paired $p=0.077$, 4:12 discordant), which is a failure to detect a difference on one run per arm and not evidence of equivalence. What the run does establish is that the repair makes the rows learnable at RL scale. It does *not* establish that reward training creates differentiated skill content: over 150 epochs the rows move 0.29% in mutually near-orthogonal directions, leaving the library’s pairwise-

	row norm	storage	survives	Δ CE	outcome
A1	as baked	bf16	0.0%	+0.002	frozen
A2	as baked	fp32	94.2%	+0.001	frozen
A3	rescaled	bf16	0.7%	+0.002	frozen
A4	rescaled	fp32	100%	-0.020	learns

Table 4: 2×2 row-training diagnostic (300 steps, only the 352 rows trainable). “Survives” is the fraction of the Adam update that outlives the storage boundary. Activation-scale norms starve the rows (A1, A2) and bfloat16 storage erases their updates (A1, A3); only the power-of-two rescaling and fp32 master storage together (A4) make updates survive and move held-out CE.

cosine structure and per-entry distinctiveness unchanged to four decimal places.

What the frozen rows carry. If the rows never effectively moved, what do they contribute? Figure 2 (left) perturbs them at evaluation. Removing the skill tokens from the prompt entirely costs 21 points (69.3 vs. 90.0, discordant 5:34, paired $p=2.4 \times 10^{-6}$): the conditioning is load-bearing. Corrupting the content while keeping the tokens is *worse than removing it*: zeroed and norm-matched Gaussian rows both land at 58.6, about 11 points below absence ($p=0.017$, $p=0.020$). What the conditioning does *not* require is any of the structure that makes it a library. Permuting the skill-to-row assignment with a derangement changes nothing detectable (92.1, 9:6, $p=0.61$). Substituting a library from an *independent* Stage-1 draw, whose per-skill vectors are nearly orthogonal to the deployed ones (cosine 0.20), leaves the checkpoint at 92.9 and 90.7 over two evaluations ($p=0.39$, $p=1.0$). And collapsing the library entirely, so every entry receives the same code and the injected content no longer depends on retrieval, also changes nothing detectable: 91.4 vs 90.0 (9:7, $p=0.80$), while still beating removal by 22.1 points ($p=3.4 \times 10^{-7}$). We therefore detect no contribution from the identity of the selected code, nor from having distinct per-entry codes: 44 distinct codes are worth no more than one repeated 44 times. The retriever and the conditioning dose remain active throughout, so what is undetectable is the *content distinction* between codes, not retrieval itself.

What *does* matter is identified by one further control. We drew a bank from a Gaussian matched to the deployed library’s first and second moments per latent position, so that mean direction, covariance and hence anisotropy are reproduced while no entry carries any information about which skill it represents (audit in Appendix G: pairwise cosine 0.716 vs 0.709, mean-direction cosine 0.997). That bank scores 92.1, with no detectable difference from the deployed library (discordant 8:5, $p=0.58$), and 33.6 points above an isotropic bank matched on per-row norm alone (49:2, $p=1.2 \times 10^{-12}$). This is a compatibility result, not a causal one: a surrogate parameterized by the bank’s own moments retains whatever the actor reads, without saying which property that is.

These results bound the mechanism without identifying it. Per-entry content is *not necessary* at the $\sim \pm 7$ -point resolution the discordant counts afford: nothing distinguish-

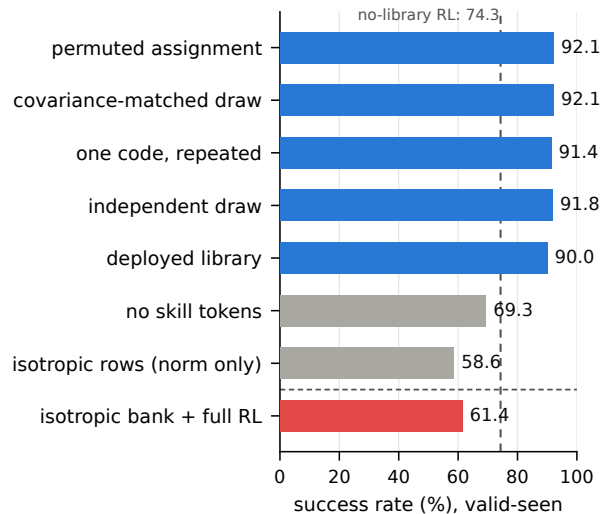


Figure 2: What the installed rows carry (*valid-seen*, $n=140$). Above the dashed line, evaluation-time interventions on the trained checkpoint: the conditioning helps regardless of code assignment, training draw, distinctness, or per-entry content, while removal costs 21 points and isotropic norm-matched rows are worse than removal. Below, the retrained control from such a bank does not recover. Every working substitution was built from the deployed bank or its statistics, so no cell isolates a cause (§4.5, Appendix G).

ing skill i ’s code from skill j ’s survives as a measurable contribution. Matching magnitude alone was not enough, which is what the retrained random bank of §4.5 was missing. Consistent with this, the deployed codes are strongly anisotropic (71.5% of each code’s energy along one shared direction) and the independent draw equally so along a *different* direction (cosine 0.24), so no direction is privileged; Appendix G tabulates every bank. Stated as measured: this actor consumes a *soft prefix drawn from the trained codes’ distribution*, and is indifferent to which entry, which draw, how many distinct codes, and to per-entry content. We tested one alternative distribution and one degenerate one, not the space between, so this separates our conditions rather than identifying a minimal sufficient one. A leave-one-out probing decoder is consistent with the same picture, finding the rows category-legible (diagnostic-word recovery 18/27 vs. 1/27 for permuted controls) but far from verbatim-decodable (Appendix D, exploratory).

4.4 What a Compressed Library Requires

A text skill means the same thing wherever it is pasted; a soft skill is a vector some encoder produced. Injecting 60 additional entries at evaluation costs 2.9 points when they come from the trained Composer that built the library (87.1 vs. 90.0, $p=0.45$) and is catastrophic when the same texts are encoded by the same architecture with a mismatched pretrained body (2.1, i.e. 67 points below running with no skills at all), with dose-monotone damage in between (8 mismatched entries leave 52.1, 30 leave 17.1). Outputs

from the tested mismatched encoder were destructive, but that bank also differs in norm (~ 264 vs ~ 235), so encoder provenance remains confounded with row norm and geometry; Appendix F gives the dose series.

4.5 Training From Random Rows

The interventions above are evaluation-time, and cannot separate a policy that exploits trained conditioning from one that co-adapted to any stable soft scaffold. That requires running the whole recipe from a bank carrying no skill content: an isotropic norm-matched Gaussian library, installed with the same bake procedure and trained 150 epochs under the same nominal GRPO hyperparameters. The intended contrast is the installed vectors; its rows were functionally unchanged, in the same quantization-limited regime as the flagship’s. The endpoint is far below the flagship on both splits: 61.4 *valid-seen* (86/140) and 52.2 *valid-unseen* (70/134), against 90.0 and 91.8 for the trained library on the same protocol. The comparisons are paired and almost perfectly one-directional (*valid-seen* discordant 3:43, exact McNemar $p=4.6 \times 10^{-10}$; *valid-unseen* 0:53, $p<10^{-15}$). It also lands below the no-library run (74.3/61.2), and its episodes are long rather than malformed (mean 35.3 steps against 18.5). Under the reading we fixed in advance, this is the first branch: the tested learned latent distribution outperforms a norm-matched random bank, and full-actor GRPO does not learn to exploit that bank within 150 epochs. This does not attribute the gap to semantic content, since norm matching equalizes magnitude only; the covariance-matched control of §4.3 attributes it to the distribution. One scoping point: this arm used the original bake recipe for single-factor comparability, so *its rows were effectively frozen too* (drift 7×10^{-6}). It therefore shows that 150 epochs of *actor* adaptation cannot compensate for a fixed off-distribution bank, not that reward is unable to repair one. Appendix G gives the chain of custody.

5 Case Study: Can the Library Self-Evolve?

SkillRL attributes +5.5 points to dynamic skill evolution, mining new skills from *validation* failures and scoring them on the same validation slots. We reran evolution leakage-free in both representations, mining only from training batches, and found no significant contrast either way: neither in the text arm (-6.4 , $p=0.12$ *valid-seen*; -0.7 , $p=1.0$ *valid-unseen*) nor in a repaired latent arm ($+1.4$, $p=0.81$; $+3.0$, $p=0.45$), all four inside this harness’s replicate noise. We therefore draw no conclusion about whether evolution helps; what we can report is an audit of the released mechanism, whose two defects are inherited verbatim rather than introduced by our port. Mining is signal-starved: decoding strips the chat-format markers the step parser requires, so every failed trajectory degrades to ~ 200 characters of boilerplate and the generator invents skills from task descriptions rather than failures. Admission is ungated: generation fires on almost every update and acceptance requires only schema validity, with no replay scoring, novelty check or removal path, consistent with the broader finding that ungated skill generation carries near-zero marginal utility (Zhang et al. 2026b; He et al. 2026b). Both are fixable, and fixing them changed the gen-

erated content without changing the endpoint. Appendix F gives the tables, the redundancy audit, and the token costs.

6 Limitations

Threats to validity. (i) The text comparison is a released checkpoint trained under a different recipe (Table 1), so it supports no causal statement about representation. (ii) Two SkillT runs cannot establish a method-level out-of-distribution effect; ours differ by 11.9 points there. (iii) The harnesses differ in history length (10 vs. 2 in training), so each estimate carries a test from its own protocol; the sequential harness’s identical-configuration replicate differs by 13 discordant episodes, a floor below which small discordant counts mean nothing. (iv) Only the environment draw is seeded, so repeated evaluations are not bitwise reproducible. (v) Every bank that succeeds in §4.3 derives from the deployed bank or its statistics, confounding Composer provenance with the matched statistics.

Scope. One environment (ALFWorld) and one backbone (Qwen2.5-7B); we make no claim about other agents, libraries or scales.

Disclosures. (1) The RL initialization is a Stage-1 draw selected on its pre-RL *valid-seen* evaluation (§4.1): second-highest of six. (2) Our parser accepts malformed action-tag variants the strict one rejects, validated on the text anchor (87.9 unchanged) and primarily raising weak baselines. (3) The text-dynamic checkpoint is step-130 of 150. (4) The 11 mistake entries are installed in neither flagship bake, favouring the text baseline. (5) Our first latent dynamic attempt was contaminated by an encoder-provenance defect of our own, so we report the repaired rerun. (6) Skill-token IDs are suppressed at evaluation but not in training; the mining study’s blinded judge is an automated model. We use ALFWorld and Qwen2.5 under their licenses; no human subjects; compute and per-cell commands are documented in the artifact.

7 Conclusion

On ALFWorld, episode-level reward trains a Qwen2.5-7B actor to work from a compact bank of soft vocabulary rows instead of a retrieved text library: 90.0/86.6 on the complete evaluation sets, with the skill section shrunk from ~ 970 to ~ 140 tokens per step; a released text-skill checkpoint reads 87.9/79.9 under the same harness and anchors rather than controls that comparison. The audit of what reward actually trained is the more transferable result. The rows were effectively frozen, for a numerical reason any system installing activation-scale vectors into a vocabulary will meet, and a function-preserving repair restores learnability without reward differentiating them. We detect no contribution from code identity or distinctness, leaving the load-bearing property unidentified.

References

Berthon, A.; Astorga, N.; and van der Schaar, M. 2026. Skill Neologisms: Towards Skill-based Continual Learning. In *Proceedings of the 43rd International Conference on Machine Learning*, PMLR 306. ArXiv:2605.04970.

- Butt, N.; Kwiatkowski, A.; Labiad, I.; Kempe, J.; and Ollivier, Y. 2025. Soft Tokens, Hard Truths. arXiv:2509.19170.
- Chevalier, A.; Wettig, A.; Ajith, A.; and Chen, D. 2023. Adapting Language Models to Compress Contexts. arXiv:2305.14788.
- Deng, J.; Wei, Z.; Pang, L.; Wu, J.; Xu, S.; Duan, Z.; and Shen, H. 2026. Latent-GRPO: Group Relative Policy Optimization for Latent Reasoning. arXiv:2604.27998.
- Dorovatas, V.; Kalaj, N.; and Aljundi, R. 2026. Continual Self-Improvement with Lightweight Experiential Latent Memories. arXiv:2606.17803.
- Feng, T.; Ye, C.; Luo, T.; Xu, J.; Xu, X.; Zhang, H.; Liu, G.; and You, J. 2026. ElasticMem: Latent Memory as a Learnable Resource for LLM Agents. arXiv:2605.30690.
- Fu, M.; Xue, X.; Li, Y.; He, Z.; Huang, S.; Qu, X.; Cheng, Y.; and Yang, Y. 2026. LatentMem: Customizing Latent Memory for Multi-Agent Systems. arXiv:2602.03036.
- Ge, T.; Hu, J.; Wang, L.; Wang, X.; Chen, S.-Q.; and Wei, F. 2024. In-context Autoencoder for Context Compression in a Large Language Model. arXiv:2307.06945.
- Hao, S.; Sukhbaatar, S.; Su, D.; Li, X.; Hu, Z.; Weston, J.; and Tian, Y. 2025. Training Large Language Models to Reason in a Continuous Latent Space. arXiv:2412.06769.
- He, Z.; Li, Y.; Huang, F.; Chen, T.; Chen, S.; Li, X.; Yu, M. H.; Liu, X.; Wei, L.; Pan, L.; Zeng, K.; and Cai, X. 2026a. SIRI: Self-Internalizing Reinforcement Learning with Intrinsic Skills for LLM Agent Training. arXiv:2606.02355.
- He, Z.; Lin, H.; Han, B.; Zhu, W.; Fang, H.; Wang, B.; Zhu, X.; Li, R.; and Reimherr, M. 2026b. ReSkill: Reconciling Skill Creation with Policy Optimization in Agentic RL. arXiv:2606.01619.
- Lester, B.; Al-Rfou, R.; and Constant, N. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 3045–3059.
- Li, X.; Li, M.; Bao, K.; Ma, Y.; Wang, W.; Liu, D.; and Feng, F. 2026. SkillGraph: Skill-Augmented Reinforcement Learning for Agents via Evolving Skill Graphs. arXiv:2605.12039.
- Li, X. L.; and Liang, P. 2021. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 4582–4597.
- Li, Z.; Su, Y.; and Collier, N. 2024. 500xCompressor: Generalized Prompt Compression for Large Language Models. arXiv:2408.03094.
- Lin, H.; Kuai, Z.; Xue, E.; and Wang, L. 2026. SKILLC: Learning Autonomous Skill Internalization in LLM Agents via Contrastive Credit Assignment. arXiv:2605.27899.
- Miahi, E.; and Belilovsky, E. 2026. Understanding and Exploiting Weight Update Sparsity for Communication-Efficient Distributed RL. arXiv:2602.03839.
- Micikevicius, P.; Narang, S.; Alben, J.; Diamos, G.; Elsen, E.; Garcia, D.; Ginsburg, B.; Houston, M.; Kuchaiev, O.; Venkatesh, G.; and Wu, H. 2018. Mixed Precision Training. ICLR 2018, arXiv:1710.03740.
- Mu, J.; Li, X. L.; and Goodman, N. 2024. Learning to Compress Prompts with Gist Tokens. arXiv:2304.08467.
- Qwen Team; Yang, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Li, C.; Liu, D.; Huang, F.; Wei, H.; Lin, H.; Yang, J.; Tu, J.; Zhang, J.; Yang, J.; Yang, J.; Zhou, J.; Lin, J.; Dang, K.; Lu, K.; Bao, K.; Yang, K.; Yu, L.; Li, M.; Xue, M.; Zhang, P.; Zhu, Q.; Men, R.; Lin, R.; Li, T.; Tang, T.; Xia, T.; Ren, X.; Ren, X.; Fan, Y.; Su, Y.; Zhang, Y.; Wan, Y.; Liu, Y.; Cui, Z.; Zhang, Z.; and Qiu, Z. 2025. Qwen2.5 Technical Report. arXiv:2412.15115.
- Sastre, I.; and Rosá, A. 2026. Concept Tokens: Learning Behavioral Embeddings Through Concept Definitions. arXiv:2601.04465.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.
- Sheng, G.; Zhang, C.; Ye, Z.; Wu, X.; Zhang, W.; Zhang, R.; Peng, Y.; Lin, H.; and Wu, C. 2024. HybridFlow: A Flexible and Efficient RLHF Framework. arXiv:2409.19256.
- Shi, Y.; Chen, Y.; Lu, Z.; Miao, Y.; Liu, S.; GU, Q.; Cai, X.; Wang, X.; and Zhang, A. 2026. Skill1: Unified Evolution of Skill-Augmented Agents via Reinforcement Learning. arXiv:2605.06130.
- Shridhar, M.; Yuan, X.; Côté, M.-A.; Bisk, Y.; Trischler, A.; and Hausknecht, M. 2021. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. arXiv:2010.03768.
- Tao, X.; Teng, Y.; Fu, X.; Liu, Z.; Chen, K.; Zhao, Y.; Zhang, S.; Liu, R.; and Kong, L. 2026. SoftSkill: Behavioral Compression for Contextual Adaptation. arXiv:2606.20333.
- Wang, C.; Su, W.; Ai, Q.; Tang, Y.; Qiao, R.; Li, X.; Zhang, M.; and Liu, Y. 2026. Adaptive Multi-Resolution Procedural Knowledge Compression for Large Language Models. arXiv:2606.12203.
- Wu, Z.; Hao, Y.; and Mou, L. 2025. TokMem: One-Token Procedural Memory for Large Language Models. arXiv:2510.00444.
- Xia, P.; Chen, J.; Wang, H.; Liu, J.; Zeng, K.; Wang, Y.; Han, S.; Zhou, Y.; Zhao, X.; Chen, H.; Zheng, Z.; Xie, C.; and Yao, H. 2026. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. arXiv:2602.08234.
- Yu, A.; Zhou, C.; Xu, T.; Guo, Z.; Shan, R.; Fu, Z.; Wang, J.; Liu, W.; Yu, Y.; Zhang, W.; and Lin, J. 2026. LatentSkill: From In-Context Textual Skills to In-Weight Latent Skills for LLM Agents. arXiv:2606.06087.
- Zhang, G.; Fu, M.; and Yan, S. 2025. MemGen: Weaving Generative Latent Memory for Self-Evolving Agents. arXiv:2509.24704.
- Zhang, S.; Wang, J.; Zhou, R.; Liao, J.; Feng, Y.; Li, Z.; Zheng, Y.; Zhang, W.; Wen, Y.; Li, Z.; Xiong, F.; Qi, Y.; Tang, B.; and Wen, M. 2026a. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory. arXiv:2601.03192.

Zhang, Z.; Lin, Y.; Kuang, N. L.; Wu, L.; Li, X.; Liu, S.; and Ma, F. 2026b. Co-Evolving Skill Generation and Policy Optimization. *arXiv:2606.08755*.

Zhao, C.; Tan, Y.; He, S.; Wang, Y.; Zhao, J.; and Liu, K. 2026. Neural Procedural Memory: Empowering LLM Agents with Implicit Activation Steering. *arXiv preprint arXiv:2606.29824*.

Zhu, R.-J.; Peng, T.; Cheng, T.; Qu, X.; Huang, J.; Zhu, D.; Wang, H.; Xue, K.; Zhang, X.; Shan, Y.; Cai, T.; Kergan, T.; Kembay, A.; Smith, A.; Lin, C.; Nguyen, B.; Pan, Y.; Chou, Y.; Cai, Z.; Wu, Z.; Zhao, Y.; Liu, T.; Yang, J.; Zhou, W.; Zheng, C.; Li, C.; Zhou, Y.; Li, Z.; Zhang, Z.; Liu, J.; Zhang, G.; Huang, W.; and Eshraghian, J. 2025. A Survey on Latent Reasoning. *arXiv:2507.06203*.